```
Cookie: PHPSESSIONID=7757ADD8766d455NFJ23875JBJKBFR from=3536702
1&to48412334&amount=5000&date=05072010
```

Using an HTTP GET request, it would look like what follows:

```
GET http://fictitiousbank/transfer.cgi?from=35367021&to48412334&am
ount=5000&date=05072010 HTTP/1.1
Host: fictitiousbank
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; de; rv:1.9)
Gecko/2008052906 Firefox/3.6.2
Cookie: PHPSESSIONID=7757ADD8766d455NFJ23875JBJKBFR
```

Note: The attacker can obtain the HTTP request/response data transmitted to/from the Web server in multiple ways: using free services offered by sites like *www.web-sniffer.net* and *www.http-sniffer.com*, or by using local network sniffers on their LAN while they use the banking site themselves. Examining the captured requests and responses lets them determine the valid URLs for the Web application.

Now assume that the victim, while still logged in to the online banking site, opens another tab/window in the same browser, for a malicious site (let's call it *www.attacker.com*). This could be due to clicking on a malicious link sent to the victim via IM or e-mail. Such a link might take the victim to *www.attacker.com/checkthisout.html*, which contains the following malicious HTML code:

```
<HTML>
<IMG SRC="http://fictiousbank/transfer.cgi?from=35367021&to4841233
4&amount=5000&date=05072010" width="0" height="0">
</HTML>
```

When the victim's browser loads this page, it will try to display the specified zero-width, zero-height (thus invisible) image as well, by making a request to the URL specified in the *SRC* attribute. Since the customer is still logged in to the banking site, an amount of 5000 from account 35367021 will be transferred to account 48412334, which could belong to the attacker. The bank site sees this request as completely legitimate, since the browser's session cookie tells it that the user authenticated himself with the correct credentials! The attack above is depicted in Figure 1.

To prevent wary users or someone accidentally stumbling across the CSRF URL as the image source (if they view the source HTML of the page), the URL could be obfuscated as a seemingly innocent HTTP request. The attacker could make it look like a valid image URL such as *<img src="https://www.attacker.com/picture1.gif" width="0" height="0">*. Here, when this request for *picture1.gif* is received, *attacker.com* uses the HTTP redirect mechanism to redirect the browser to a different URL. That different URL is the Web application

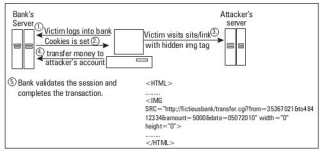

Figure 1: CSRF attack via GET

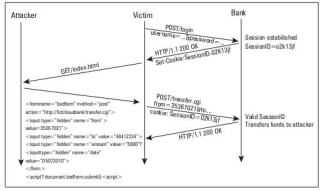

Figure 2: CSRF attack using HTTP POST

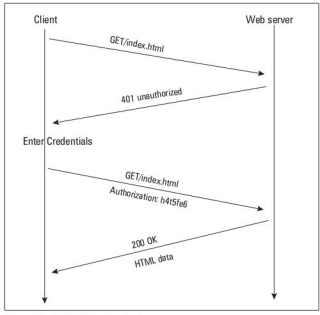

Figure 3: Typical HTTP authentication

state-changing URL—like the money-transfer action URL in this scenario.

The attack method above (*IMG SRC*) could be used if the Web application uses HTTP *GET* to submit requests. If it uses HTTP *POST* instead, then the attacker can use a hidden *IFRAME* containing an HTML form, plus JavaScript that submits it to the bank site, like this snippet:

```
<form name="badform" method="post" action="http://fictitiousbank/
```